SPECIFICATION

Calypso Terminal API

Generic Card

Version 2.0.0, 2026-10-28



Warning. The present document cancels and replaces the previous release.

© Calypso Networks Association 2026— Reproduction is prohibited without prior written agreement

Link and Contact



Website
https://calypsonet.org



Support
support@calypsonet.org

Copyright © Calypso Networks Association, https://calypsonet.org.

This specification is licensed under the **Creative Commons Attribution-NoDerivatives 4.0 International** *(CC BY-ND 4.0).*

You are free to **share** *— copy and redistribute this document in any medium or format for any purpose, even commercially — under the following terms:*

- ***Attribution*** *— You MUST give appropriate credit to the Calypso Networks Association, provide a link to the license, and indicate if changes were made. You MAY do so in any reasonable manner, but not in any way that suggests the Calypso Networks Association endorses you or your use.*
- ***NoDerivatives*** *— If you remix, transform, or build upon this document, you MAY NOT distribute the modified material.*

No additional restrictions — You MAY NOT apply legal terms or technological measures that legally restrict others from doing anything the license permits.

Implementation grant *— The* **NoDerivatives** *condition above applies to this specification* **document** *only — its text, tables and diagrams. It does* **not** *restrict the use of the technical information the document contains. The Calypso Networks Association hereby grants everyone an irrevocable, worldwide, royalty-free permission to use the information contained in this specification to design, develop, and distribute software implementations of this API, in any programming language, under the license of their choice.*

The full license text is available at https://creativecommons.org/licenses/by-nd/4.0/.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Table of Contents

1. Abstract . . . 4
2. Introduction . . . 5
2.1. Purpose . . . 5
2.2. Scope . . . 5
2.3. Conformance . . . 5
2.4. Document Conventions . . . 6
2.4.1. Naming . . . 6
2.4.2. Data types . . . 6
2.4.3. Type tables . . . 6
2.4.4. Operation tables . . . 7
2.4.5. Operation contracts . . . 7
2.4.6. Concurrency . . . 7
2.5. References and Resources . . . 7
2.5.1. Calypso References . . . 7
2.5.2. Normative References . . . 8
2.5.3. External Resources . . . 8
3. Glossary and Acronyms . . . 9
3.1. Glossary . . . 9
3.2. Acronyms . . . 9
4. Architectural Overview . . . 10
4.1. Functional positioning . . . 10
4.2. Logical namespaces . . . 10
4.3. UML class diagram . . . 10
5. API Specification . . . 12
5.1. Constants . . . 12
5.1.1. Generic Card API Properties . . . 12
5.2. API Interfaces . . . 12
5.2.1. Generic Card API Factory . . . 12
Create Generic Card Selection Extension . . . 12
Create Generic Card Transaction Manager . . . 12
5.2.2. Generic Card Selection Extension . . . 13
Add Successful Status Word . . . 13
5.2.3. Generic Card Transaction Manager . . . 13
Get Last Execution Response . . . 14
Get Last Execution Responses . . . 14
Prepare Command . . . 14
Prepare Command With ID . . . 14
Prepare Command With Max Duration . . . 15

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

1. Abstract

This document is the official technical specification of the **Terminal Generic Card API** standardised by the **Calypso Networks Association** (CNA). It defines, in a language-agnostic way (using a notation inspired by the Kotlin language), the interfaces, classes, enumerations, errors, structural relationships and behavioural requirements that any conforming implementation of the Terminal Generic Card API MUST satisfy.

The Terminal Generic Card API is part of the broader CNA **Terminal API** family. It exposes the minimal contract required to exchange raw APDUs with an ISO/IEC 7816-4 card that has been selected through the **Terminal Reader API**, without prescribing any specific card application model.

Document Status

Reference	261028-SP-CNATerminalAPI-GenericCard
Short name	CNA-TGC-API
Version	2.0.0
Revision date	2026-10-28
Editor	Calypso Networks Association
Source repository	https://github.com/calypsonet/calypsonet-terminal-genericcard-api-spec
Reference license	Creative Commons Attribution-NoDerivatives 4.0 International (CC BY-ND 4.0)

The present document specifies the 2.0.0 of the Terminal Generic Card API. It defines a single, coherent baseline against which conforming implementations are evaluated; the **Since** column indicates the version in which each member was introduced.

Revision List

This section lists the high-level changes per version. The complete, fine-grained changelog is maintained in the `CHANGELOG.md` file at the root of the source repository.

Version / Date	Modifications
v2.0.0 2026-10-28	Initial edition of this document.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

2. Introduction

2.1. Purpose

The Terminal Generic Card API defines the contract used by terminal applications that need to exchange raw APDUs with an ISO/IEC 7816-4-compliant card without relying on a card-specific application layer (e.g. Calypso). It complements the **Terminal Reader API** (CNA-TR-API) by providing:

- a card-selection extension that allows additional successful status words to be declared for the **Select Application** APDU,
- a minimal transaction manager that prepares APDU commands, attaches an optional command identifier and an optional per-command duration bound (relay-attack countermeasure), and processes them as a single batch, returning the raw responses to the application.

Conforming implementations MUST guarantee interoperability with reader implementations of the CNA-TR-API specification.

2.2. Scope

This specification covers:

- the factory used to instantiate the public types of the API,
- the card-selection extension used to enrich the selection scenario for generic cards,
- the transaction manager used to prepare and exchange APDUs with the selected card, including the per-command identifier and the per-command duration bound,
- the property holder exposing the API version.

The following topics are **out of scope**:

- the parsing of APDU payloads—the API exposes raw bytes only,
- the application-level dialog with specific card products (covered by dedicated specifications),
- the management of secure channels.

2.3. Conformance

A software product conforms to this specification if and only if:

1. it implements every interface and class defined in Chapter 5 with the operations and semantics prescribed in this document,
2. it preserves the structural relationships described in Chapter 4.

Throughout this specification, the key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY** and **OPTIONAL** are to be interpreted as described in RFC 2119 and RFC 8174 when, and only when, they appear in all capitals.

In conformance with RFC 2119, **SHALL** is equivalent to **MUST**; this specification uses **MUST** exclusively in order to avoid ambiguity.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

2.4. Document Conventions

2.4.1. Naming

Type names follow `UpperCamelCase`; operation, parameter and enumeration-member names follow `lowerCamelCase` except for enumeration values which use `UPPER_SNAKE_CASE`. Names defined by this specification are reproduced as-is in the UML class diagram (Section 4.3).

2.4.2. Data types

The following type names, whose notation is inspired by the Kotlin language, are used in operation signatures and are mapped, in each binding, to the closest equivalent native type. Implementations MUST preserve the semantic intent rather than the syntactic form.

Type	Description	Typical bindings
`String`	Sequence of characters, UTF-8 encoded by default.	`String`, `string`, `str`.
`Boolean`	Two-state truth value.	`boolean`, `bool`.
`Byte`	Signed 8-bit value.	`byte`, `i8`.
`Int`	Signed 32-bit integer.	`int`, `Int32`, `i32`.
`Long`	Signed 64-bit integer.	`long`, `Int64`, `i64`.
`ByteArray`	Ordered sequence of octets.	`byte[]`, `[u8]`, `bytes`.
`List<T>`	Ordered collection of `T` values, may contain duplicates.	`List<T>`, `Vec<T>`.
`T?`	Nullable value: either a `T` value or `null`.	`Optional<T>`, `T?`, `Option<T>`.
`null`	Absence of value, the alternative held by a nullable type.	`null`, `nil`, `None`.
`Self`	Own type of the receiver: a fluent operation returns the instance it was called on.	`Self`, `T` (recursive generic).
`Unit`	Indicates that an operation returns no value.	`void`, `Unit`, `()`.

2.4.3. Type tables

Each interface, class, enumeration and error defined in this document is described by a table of the form:

Name	The type's normalized name.
Kind	The category of the type (interface, class, enumeration, error, etc.).
Namespace	The namespace in which the type is defined.
Extends	The type extended by this type, when applicable.
Implements	The interface implemented by this type, when applicable.
Since	The version of the API in which the type was introduced.
Purpose	A normative description of the type's responsibility.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

The **Purpose** row states, in one or two sentences, the responsibility of the type and the way an instance is obtained. It is meant to be scanned, not read: any content that requires structure or depth—rules, lists, notes, value tables or examples—is placed as prose below the table.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

2.4.4. Operation tables

Each operation defined in this document is described by a table of the form:

Signature	The operation's signature.
Since	The version of the API in which the operation was introduced.
Description	A normative description of the operation's behaviour.
Parameters	A description of each input parameter.
Returns	A description of the returned value, if any.
Pre-conditions	The conditions the caller MUST satisfy before invoking the operation, each prefixed by its nature.
Errors	A list of error conditions a correct caller must handle, each mapped to a specific error type.
See also	Cross-references to related operations or types, each a hyperlink to the corresponding table. Present only when applicable.

2.4.5. Operation contracts

To avoid repetition, the following rules apply to every operation table in Chapter 5 and are therefore not restated for each operation:

- **Pre-conditions.** The **Pre-conditions** row states what the caller MUST guarantee before invoking the operation. Violating one is a contract error, never listed in the **Errors** row, which describes only the outcomes a correct caller must handle. Each entry is prefixed by its nature, which determines the error raised when the condition is not met:
  - **Argument**—an illegal argument error. Applies to any invalid argument, including values bounded by a card or SAM protocol.
  - **Range**—an index out of bounds error. Applies when the argument designates a position within a collection or memory image exposed by the API.
  - **State**—an illegal state error.
  - **Capability**—an unsupported operation error.
- **Non-`null` results.** Unless the return type is marked nullable (`T?`) or the **Returns** row states otherwise, an operation returns a non-`null` value.
- **Collections.** Unless the return type is marked nullable (`T?`), an operation whose return type is a collection (e.g. `List`, `Set`, `Map`) returns an empty collection rather than `null`.
- **Fluent composition.** Builder-style operations return the current instance so that calls can be chained.

2.4.6. Concurrency

Unless explicitly stated otherwise, the stateful types defined by this specification are **not** thread-safe. A given instance MUST be accessed from a single thread at a time; concurrent use of the same instance without external synchronisation results in undefined behaviour. Distinct instances MAY be used concurrently.

2.5. References and Resources

2.5.1. Calypso References

References to CNA Terminal APIs designate the indicated **major** version; any later release within the same major is backward-compatible per that API's evolution policy.

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Reference	Document
[CNA-TR-API]	*CNA Terminal API — Reader* (SP-CNATerminalAPI-Reader), version 3.0, Calypso Networks Association.

2.5.2. Normative References

Reference	Document
[RFC 2119]	*Key words for use in RFCs to Indicate Requirement Levels*, S. Bradner, March 1997.
[RFC 8174]	*Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, B. Leiba, May 2017.
[ISO/IEC 7816]	*Identification cards — Integrated circuit cards*.

2.5.3. External Resources

Resource	Link
Calypso Networks Association	https://calypsonet.org
Terminal APIs website	https://terminal-api.calypsonet.org
Terminal APIs documentation	https://docs.terminal-api.calypsonet.org

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

3. Glossary and Acronyms

3.1. Glossary

Term	Definition
APDU	Application Protocol Data Unit, the message format used between the terminal and the card, as defined in ISO/IEC 7816-3.
Card Reader	Hardware or software component capable of detecting cards and exchanging data with them.
Command Identifier	Application-supplied integer attached to a prepared APDU, used to retrieve the corresponding response or to identify the failing command in case of error.
Relay Attack	Software-level attack consisting in interposing a relay between the card and the terminal to redirect APDU exchanges to a distant card.

3.2. Acronyms

Abbreviation	Expansion
AID	Application IDentifier
APDU	Application Protocol Data Unit
CNA	Calypso Networks Association
ISO	International Organization for Standardization
SW	Status Word

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

4. Architectural Overview

4.1. Functional positioning

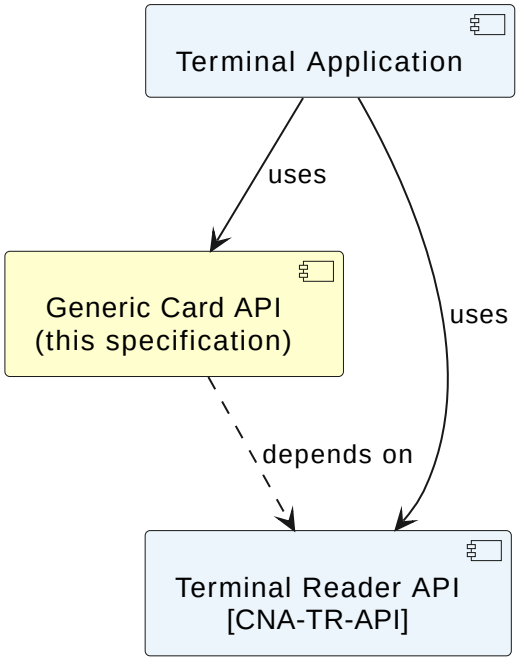


Figure 1. Terminal Generic Card API — Architecture Overview

4.2. Logical namespaces

Namespace	Role	Summary
`genericcard`	Public API	Factory, selection extension and transaction manager. Properties.

4.3. UML class diagram

The following UML class diagram provides a visual representation of the types and relationships defined by this specification:

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

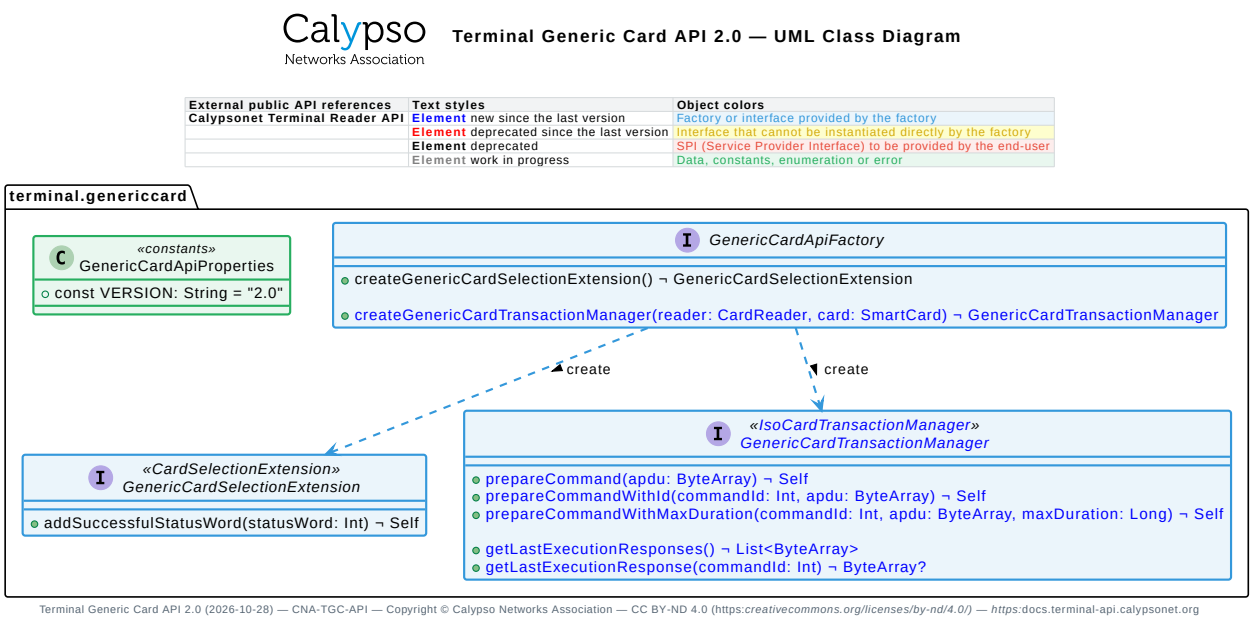


Figure 2. Terminal Generic Card API — UML Class Diagram

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

5. API Specification

5.1. Constants

5.1.1. Generic Card API Properties

Name	`GenericCardApiProperties`
Kind	Constants
Namespace	`genericcard`
Since	1.0
Purpose	Exposes the immutable properties of the Terminal Generic Card API.

Name	Type	Since	Description
`VERSION`	`String`	1.0	String representation of the version of the API implemented by the conforming binding. The value is a dotted decimal of the form `MAJOR.MINOR`; it is `"2.0"` for this revision of the specification.

5.2. API Interfaces

5.2.1. Generic Card API Factory

Name	`GenericCardApiFactory`
Kind	Interface
Namespace	`genericcard`
Since	1.0
Purpose	Factory used by the application to obtain instances of the public types provided by the API.

The `GenericCardApiFactory` is the single entry point through which the application instantiates the public types of the API. It is provided as a single instance whose accessors each return a fresh, fully initialised object.

Create Generic Card Selection Extension

Signature	`createGenericCardSelectionExtension() → GenericCardSelectionExtension`
Since	1.0
Description	Returns a new, freshly initialised `GenericCardSelectionExtension`.
Returns	A `GenericCardSelectionExtension`.
Pre-conditions	None.
Errors	None.

Create Generic Card Transaction Manager

Signature	`createGenericCardTransactionManager(`
	`  reader: CardReader,`
	`  card: SmartCard`
	`) → GenericCardTransactionManager`

Since	1.0

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

Description	Returns a new, freshly initialised `GenericCardTransactionManager`, bound to the supplied reader and to the initial card data provided by the selection process.
Parameters	`reader` (`CardReader` (CNA-TR-API)) — the reader through which the card is reached. `card` (`SmartCard` (CNA-TR-API)) — the initial card data provided by the selection process.
Returns	A `GenericCardTransactionManager`.
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

5.2.2. Generic Card Selection Extension

Name	`GenericCardSelectionExtension`
Kind	Interface
Namespace	`genericcard`
Extends	`CardSelectionExtension` (CNA-TR-API)
Since	1.0
Purpose	Extends the `CardSelectionExtension` interface defined by the **Terminal Reader API** to expose a way to declare additional successful status words for the **Select Application** APDU. An instance is obtained via `GenericCardApiFactory.createGenericCardSelectionExtension`.

Add Successful Status Word

Signature	`addSuccessfulStatusWord(statusWord: Int) → Self`
Since	1.0
Description	Adds a status word to the list of those considered successful for the **Select Application** APDU. **Note:** the list initially contains the standard successful status word `9000h`; each call adds to the existing set and never removes that implicit entry.
Parameters	`statusWord` — the status word to add.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `statusWord` is in the range `[0x0000, 0xFFFF]`.
Errors	None.

5.2.3. Generic Card Transaction Manager

Name	`GenericCardTransactionManager`
Kind	Interface
Namespace	`genericcard`
Extends	`IsoCardTransactionManager` (CNA-TR-API)
Since	1.0
Purpose	Provides the basic operations required to prepare, identify and process APDU exchanges with an ISO/IEC 7816-4 card. Extending `IsoCardTransactionManager` grants access to the multi-channel cast `asMultichannelCardTransactionManager()` defined by CNA-TR-API when the underlying card supports multi-channel. An instance is obtained via `GenericCardApiFactory.createGenericCardTransactionManager`.

The `GenericCardTransactionManager` uses a **prepare/process** model inherited from CNA-TR-API: the application prepares APDU commands with `prepareCommand`, `prepareCommandWithId` or

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement

`prepareCommandWithMaxDuration` and processes them through the inherited `processCommands()` operation. Each prepared command MAY carry an optional **command identifier** (`commandId`) supplied by the application, used to retrieve the corresponding response or to identify the failing command in case of error. Each prepared command MAY also carry an optional **maximum duration** (`maxDuration`, in microseconds, the effective resolution of the measurement depending on the implementation) used to control the APDU exchange execution time at the individual-command level—for example to detect a relay attack.

Get Last Execution Response

Signature	`getLastExecutionResponse(commandId: Int) → ByteArray?`
Since	2.0
Description	Returns the response of the command identified by `commandId`. If several prepared commands share the same identifier, the response of the **last** matching command is returned. The value returned for a given identifier is the same as the corresponding element of `getLastExecutionResponses`.
Parameters	`commandId`—the application-supplied identifier of the command whose response is requested.
Returns	The non-empty response bytes, or `null` if no command with the supplied identifier produced a response.
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Get Last Execution Responses

Signature	`getLastExecutionResponses() → List<ByteArray>`
Since	2.0
Description	Returns the responses collected during the last call to `processCommands()`, in the order in which the commands were prepared. This operation does not alter the internal state of the manager: the list of responses remains available until `processCommands()` is called again, at which point it is replaced by the new set of responses.
Returns	A `List<ByteArray>` representing the command responses, in the order in which the commands were sent; empty if no command has been executed.
Pre-conditions	None.
Errors	None.

Prepare Command

Signature	`prepareCommand(apdu: ByteArray) → Self`
Since	2.0
Description	Prepares an APDU command without command identifier nor duration bound.
Parameters	`apdu`—a non-empty `ByteArray` containing the raw APDU command.
Returns	The current instance (`Self`).
Pre-conditions	**Argument**—every parameter is non-`null`.
Errors	None.

Prepare Command With ID

© Calypso Networks Association 2026—Reproduction is prohibited without prior written agreement

Signature	`prepareCommandWithId(`
	`  commandId: Int,`
	`  apdu: ByteArray`
	`) → Self`

Since	2.0
Description	Prepares an APDU command with an application-supplied identifier. The identifier MAY later be used to retrieve the corresponding response via `getLastExecutionResponse`.
Parameters	`commandId` — the application-supplied identifier of this command. `apdu` — a non-empty `ByteArray` containing the raw APDU command.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`.
Errors	None.

Prepare Command With Max Duration

Signature	`prepareCommandWithMaxDuration(`
	`  commandId: Int,`
	`  apdu: ByteArray,`
	`  maxDuration: Long`
	`) → Self`

Since	2.0
Description	Prepares an APDU command with an application-supplied identifier and a maximum tolerated exchange duration. The effective duration of the corresponding APDU exchange is measured, and an `InvalidCardResponse` (CNA-TR-API) is raised if it exceeds the declared bound; the error message identifies the offending command.
Parameters	`commandId` — the application-supplied identifier of this command. `apdu` — a non-empty `ByteArray` containing the raw APDU command. `maxDuration` — the maximum tolerated duration of the APDU exchange, in microseconds.
Returns	The current instance (`Self`).
Pre-conditions	**Argument** — every parameter is non-`null`. **Argument** — `maxDuration` is strictly positive.
Errors	None at preparation time. The duration check is performed at processing time.

© Calypso Networks Association 2026 — Reproduction is prohibited without prior written agreement